Task 1 - Part A: Whitening and Autocovariance Matrices

Zhuoyan Jiang

2026-10-11

Overview

This report implements Part A of the assignment: the preprocessing and matrix computations required by the time-series version of the ladle estimator of Nordhausen and Virta (2018). The report has two parts.

- **Section 1** defines the generic functions: centering, whitening, autocovariance estimation, symmetrization, and the AMUSE / SOBI diagonalizers.
- **Section 2** applies these functions to the course data `timeseries.dat` and verifies the implementation numerically.

All computations are implemented by us. The only ready-made routines used are `JADE::rjd()` for joint diagonalization (as permitted by the assignment) and `JADE::SOBI()` for an independent verification step. The functions defined here are written so that they can be re-applied inside a bootstrap loop in Part C, where each bootstrap sample must be re-centered, re-whitened, and re-diagonalized.

---

1. Generic functions

1.1 Conventions

We write T for the sample size (called n in the assignment) and p for the dimension. **Input convention:** every function that takes data expects a numeric $T \times p$ matrix whose rows are the time points, in time order, and whose columns are the variables. This matters in Part C, where every bootstrap sample is again a $T \times p$ matrix: a transposed input would give a wrong result without any error message. The following conventions are used consistently.

- **Centering.** Column-wise subtraction of the sample mean:

$$\bar{x} = \frac{1}{T}\sum_{t=1}^{T} x_t, \qquad \tilde{x}_t = x_t - \bar{x}.$$

- **Scaling.** No additional marginal standardization (we do not divide each column by its standard deviation first). Whitening uses the full sample covariance matrix, not only the column standard deviations, so it removes both the differences in scale and the contemporaneous correlation between columns. All scaling is absorbed into the whitening step. The whitening matrix is the symmetric inverse square root

$$\hat{\Sigma}^{-1/2} = V D^{-1/2} V^\top,$$

  computed from the eigendecomposition $\hat{\Sigma} = V D V^\top$ of the sample covariance.

- **Lagging.** For a lag $\tau \geq 1$, the pairs $(x_t, x_{t+\tau})$, with $t = 1, \ldots, T-\tau$, are used. No circular wrap-around in Part A.

- **Normalization.** The $1/T$ normalization is used throughout for the sample covariance and the autocovariance matrices:

$$\hat{\Sigma} = \frac{1}{T}\sum_{t=1}^{T} \tilde{x}_t \tilde{x}_t^\top, \qquad \hat{\Sigma}_\tau = \frac{1}{T}\sum_{t=1}^{T-\tau} \hat{x}_t^{st} \hat{x}_{t+\tau}^{st\top}.$$

  Here $\hat{x}_t^{st}$ denotes the *whitened* series, so the autocovariance is estimated on the whitened data. The $1/T$ normalization is the same convention as R's `acf(..., type = "covariance", demean = FALSE)`.

- **Symmetrization.** Under the model, Σ_τ is symmetric. The finite-sample estimate $\hat{\Sigma}_\tau$ is in general not symmetric, and its antisymmetric part is purely sampling noise. We remove that part:

$$\tilde{\Sigma}_\tau = \frac{1}{2}\left(\hat{\Sigma}_\tau + \hat{\Sigma}_\tau^\top\right).$$

  Under the model, symmetrization removes sampling noise and does not remove signal.

- **Definition of Λ and ordering.** Let $\tilde{\Sigma}_\tau$ be the symmetrized autocovariance matrix of the whitened series. The vector `Lambda` returned by our functions contains the diagonal elements of the matrix Λ in the paper, that is, the values λ_j^2:

  – AMUSE (one lag τ): $\lambda_j^2 = \left(u_j^\top \tilde{\Sigma}_\tau u_j\right)^2$.
  – SOBI (lag set $\mathcal{T}$): $\lambda_j^2 = \sum_{\tau\in\mathcal{T}} \left(v_j^\top \tilde{\Sigma}_\tau v_j\right)^2$.

  Here u_j and v_j are the columns of the estimated orthogonal matrix that belong to λ_j^2. In both cases the columns are ordered so that $\lambda_0^2 \geq \lambda_1^2 \geq \cdots \geq \lambda_{p-1}^2$. The paper indexes these values from 0 to $p-1$, so `Lambda[k + 1]` in R is λ_k^2. Part C uses the same sorted `Lambda` and the same column order. In the theory the orthogonal matrix is called U. In the code it is called `U` for AMUSE and `V` for SOBI (`V` is the name used by `rjd()`).

1.2 Why whitening

In the second-order source separation (SOS) model

$$x_t = \mu + \Omega z_t, \qquad \mathrm{Cov}(z_t) = I_p,$$

the covariance of x_t is $\Sigma = \Omega\Omega^\top$, so the matrix $\Sigma^{-1/2}\Omega$ is orthogonal. The whitened series is therefore

$$x_t^{st} = \Sigma^{-1/2}(x_t - \mu) = U z_t, \qquad U^\top U = I_p.$$

This has three consequences.

1. The unknown mixing matrix Ω reduces to an **orthogonal** matrix U.
2. The lag-τ autocovariance becomes

$$\Sigma_\tau = U\, D_\tau\, U^\top, \qquad D_\tau = \mathrm{Cov}(z_t, z_{t+\tau}),$$

   If the component processes are mutually independent (assumptions (A1) and (A2) of Nordhausen and Virta, 2018), or at least uncorrelated across components at every lag, then D_τ is diagonal. The condition $\mathrm{Cov}(z_t) = I_p$ alone is not enough, because it only describes the components at the same time point. The diagonal form of D_τ is the condition that lets all the whitened lagged autocovariance matrices share the same diagonalizing matrix U. So a single lag can be diagonalized by `eigen()` (AMUSE), and a set of lags can be jointly diagonalized by an orthogonal matrix (SOBI).
3. Whitening removes scale and contemporaneous correlation; what is left in $\tilde{\Sigma}_\tau$ is the serial dependence that distinguishes signal components from white noise.

1.3 Functions

```r
center_series <- function(X) {
  mu_hat <- colMeans(X)
  Xc     <- sweep(X, 2, mu_hat, FUN = "-")
  list(mu = mu_hat, Xc = Xc)
}

whiten_series <- function(Xc) {
  Tn <- nrow(Xc)
  p  <- ncol(Xc)

  S   <- crossprod(Xc) / Tn                # (1/T) * Xc^T Xc
  eig <- eigen(S, symmetric = TRUE)
  stopifnot(min(eig$values) > 1e-10 * max(eig$values))   # covariance matrix must not be singular

  D_inv_sqrt <- diag(1 / sqrt(eig$values), nrow = p)
  S_inv_sq   <- eig$vectors %*% D_inv_sqrt %*% t(eig$vectors)

  Xw <- Xc %*% S_inv_sq

  list(S = S, eig = eig, S_inv_sq = S_inv_sq, Xw = Xw)
}

# Lag-tau autocovariance of the whitened series: (1/T) * sum_t x_t x_{t+tau}'
autocov_raw <- function(Xw, tau) {
  stopifnot(tau >= 1, tau < nrow(Xw))     # valid lag
  Tn <- nrow(Xw)
  A  <- Xw[1:(Tn - tau), , drop = FALSE]
  B  <- Xw[(1 + tau):Tn, , drop = FALSE]
  crossprod(A, B) / Tn
}

symmetrize <- function(M) {
  0.5 * (M + t(M))
}

# AMUSE: Lambda_j = (u_j' Sigma_tau u_j)^2 for ONE lag, sorted decreasingly
amuse_fit <- function(Sigma_tau_sym) {
  eig <- eigen(Sigma_tau_sym, symmetric = TRUE)    # u_j' Sigma_tau u_j = eig$values[j]
  ord <- order(eig$values^2, decreasing = TRUE)    # sort by lambda^2, not by signed eigenvalue
  list(
    U         = eig$vectors[, ord, drop = FALSE],
    diag_vals = eig$values[ord],
    Lambda    = eig$values[ord]^2
  )
}

sobi_fit <- function(X, lags = 1:12) {
  cen <- center_series(X)
  wht <- whiten_series(cen$Xc)
  Xw  <- wht$Xw
  p   <- ncol(Xw)

  S <- array(NA_real_, dim = c(p, p, length(lags)))
```

```
  for (k in seq_along(lags)) {
    S[, , k] <- symmetrize(autocov_raw(Xw, lags[k]))
  }

  V <- rjd(S)$V

  # SOBI: Lambda_j = sum over lags of (v_j' Sigma_tau v_j)^2, then sorted decreasingly
  Lambda_raw <- Reduce(`+`, lapply(seq_along(lags), function(k) {
    diag(t(V) %*% S[, , k] %*% V)^2
  }))
  ord <- order(Lambda_raw, decreasing = TRUE)

  list(V = V[, ord, drop = FALSE], Lambda = Lambda_raw[ord], Xw = Xw, S = S,
       W = wht$S_inv_sq, mu = cen$mu)
}
```

---

2. Application to `timeseries.dat`

2.1 Data

```
ts_data <- as.matrix(read.table("timeseries.dat", header = FALSE))

cat("Dimension of the data (T x p):", dim(ts_data))
```

```
## Dimension of the data (T x p): 1000 10
```

```
anyNA(ts_data)      # any missing values?
```

```
## [1] FALSE
```

The data have $T = 1000$ time points and $p = 10$ variables, so the input convention of Section 1.1 holds (rows are time points and $T > p$). There are no missing values.

2.2 Centering and whitening

```
cen    <- center_series(ts_data)
mu_hat <- cen$mu
Xc     <- cen$Xc

wht <- whiten_series(Xc)
S   <- wht$S
Xw  <- wht$Xw

cat("Column means after centering (should be approximately zero):")
```

```
## Column means after centering (should be approximately zero):
```

```
print(colMeans(Xc))
```

```
##            V1            V2            V3            V4            V5
## -2.459144e-17  3.352874e-17  5.996981e-15  5.506706e-17 -1.078998e-17
##            V6            V7            V8            V9           V10
##  2.358114e-16 -3.828049e-16  4.718448e-18  3.632650e-16 -5.206946e-17
```

```r
cat("Eigenvalues of the sample covariance matrix:")
```

```
## Eigenvalues of the sample covariance matrix:
```

```r
print(wht$eig$values)
```

```
##  [1] 4.313670 3.783830 3.316384 2.710873 2.551159 2.333506 1.822926 1.448601
##  [9] 1.250509 1.003756
```

The column means after centering are zero up to rounding error. All eigenvalues of the sample covariance matrix are positive (the smallest is about 1.0), so the inverse square root $\hat{\Sigma}^{-1/2}$ exists.

2.3 Numerical verification of whitening

The whitened sample covariance should be close to I_p:

$$\frac{1}{T}\sum_{t=1}^{T} \hat{x}_t^{st}\hat{x}_t^{st\top} \approx I_p.$$

Because we use the $1/T$ normalization, the check is against I_p. Had we used R's `cov()` (which divides by $T-1$), the result would be $\frac{T-1}{T} I_p \approx 0.999 I_p$, not I_p.

```r
S_white <- crossprod(Xw) / nrow(Xw)
err_F   <- sqrt(sum((S_white - diag(ncol(Xw)))^2))
off_max <- max(abs(S_white - diag(ncol(Xw))))

cat("Frobenius distance ||S_white - I||_F =", signif(err_F, 4))
```

```
## Frobenius distance ||S_white - I||_F = 9.655e-15
```

```r
cat("Max absolute deviation from I        =", signif(off_max, 4))
```

```
## Max absolute deviation from I        = 2.998e-15
```

Both quantities are at machine precision, confirming the whitening has the intended effect on the sample covariance matrix.

2.4 Autocovariance matrices

We estimate the autocovariance matrices of the whitened series at lags 1 to 12 (the SOBI lag set used in Nordhausen and Virta, 2018) and symmetrize them as described in Section 1.1. The matrix at lag 1 is used by AMUSE. `sobi_fit()` recomputes all twelve matrices from the raw data, so that it can be applied to bootstrap samples in Part C.

```r
lags <- 1:12
p    <- ncol(Xw)

Sigma_sym_list <- lapply(lags, function(tau) {
  symmetrize(autocov_raw(Xw, tau))
})
```

2.5 Independent check of the autocovariances

We compare the matrices with R's multivariate `acf(..., type = "covariance", demean = FALSE)`, which computes the same $1/T$ sums of lagged products, including the cross terms between different columns. Only the symmetrized matrices are used later, so both sides are symmetrized before the comparison. This also means that the check does not depend on the order of the two indices in the array returned by `acf()`.

```r
a <- acf(Xw, lag.max = 12, type = "covariance",
         plot = FALSE, demean = FALSE)$acf

diffs <- sapply(lags, function(k) {
  max(abs(Sigma_sym_list[[k]] - symmetrize(a[k + 1, , ])))
})

cat("Max absolute difference versus acf() over all lags and entries:",
    signif(max(diffs), 4))
```

```
## Max absolute difference versus acf() over all lags and entries: 0
```

The difference is zero up to rounding error, so the autocovariance matrices agree with R's own computation.

2.6 AMUSE (two-matrix / eigenvalue case)

AMUSE uses a single lag τ and finds an orthogonal matrix U_τ that diagonalizes $\tilde{\Sigma}_\tau$:

$$U_\tau^\top \tilde{\Sigma}_\tau U_\tau = \text{diagonal}, \qquad \Lambda = \operatorname{diag}\left(U_\tau^\top \tilde{\Sigma}_\tau U_\tau\right)^2.$$

The columns of U_τ are reordered so that the corresponding Λ values are decreasing. `amuse_res$Lambda` contains $(\lambda_0^2, \ldots, \lambda_{p-1}^2)$ as defined in Section 1.1. We use $\tau = 1$.

```r
tau_amuse <- 1
amuse_res <- amuse_fit(Sigma_sym_list[[which(lags == tau_amuse)]])

cat("AMUSE at lag tau =", tau_amuse)
```

```
## AMUSE at lag tau = 1
```

```r
cat("Lambda (decreasing):")
```

```
## Lambda (decreasing):
```

```r
print(round(amuse_res$Lambda, 4))
```

```
##  [1] 0.0558 0.0423 0.0146 0.0084 0.0052 0.0025 0.0019 0.0016 0.0011 0.0000
```

2.7 SOBI (joint diagonalization via `JADE::rjd()`)

SOBI uses the lag set $\mathcal{T} = \{1, \ldots, 12\}$ and jointly diagonalizes $\tilde{\Sigma}_{\tau_1}, \ldots, \tilde{\Sigma}_{\tau_K}$ by an orthogonal matrix $U_\mathcal{T}$:

$$\Lambda = \sum_{\tau \in \mathcal{T}} \operatorname{diag}\left(U_\mathcal{T}^\top \tilde{\Sigma}_\tau U_\mathcal{T}\right)^2.$$

We use the function `sobi_fit()` defined in Section 1.3, which will also be used in Part C for each bootstrap sample.

```r
sob <- sobi_fit(ts_data, lags = 1:12)
V   <- sob$V

cat("SOBI Lambda (decreasing):")
```

```
## SOBI Lambda (decreasing):
```

```r
print(round(sob$Lambda, 4))
```

```
##  [1] 0.3986 0.3419 0.1397 0.0335 0.0310 0.0247 0.0246 0.0198 0.0173 0.0125
```

```r
cat("Orthogonality error ||V'V - I||_max =",
    signif(max(abs(crossprod(V) - diag(ncol(V)))), 4))
```

```
## Orthogonality error ||V'V - I||_max = 2.22e-15
```

`sob$Lambda` contains $(\lambda_0^2, \ldots, \lambda_{p-1}^2)$ as defined in Section 1.1, so Part C can use it directly. The orthogonality error is at machine precision, so V is orthogonal.

As an independent check, we compare our components with those of the ready-made `JADE::SOBI()`. This function is used only for this check. Because the sign and the order of components are not identifiable, we compare them by absolute correlation. For each of our components we report the best match and the second-best match among the `JADE::SOBI()` components.

```r
ref <- JADE::SOBI(ts_data, k = 12)
C   <- abs(cor(sob$Xw %*% V, ref$S))

best   <- apply(C, 1, max)
second <- apply(C, 1, function(r) sort(r, decreasing = TRUE)[2])

cat("Smallest best-match correlation:", signif(min(best), 4))
```

```
## Smallest best-match correlation: 0.9996
```

```r
cat("Largest second-best correlation:", signif(max(second), 4))
```

```
## Largest second-best correlation: 0.02745
```

Each of our components has one match with absolute correlation close to 1, and all other correlations are close to 0. Our solution and `JADE::SOBI()` therefore produce the same components up to sign and order.

---

3. Summary of Part A

- **Conventions.** Column centering; no additional scaling; positive lags only; $1/T$ normalization (same as R's `acf(type = "covariance", demean = FALSE)`); symmetrization of the sample autocovariance matrices.
- **Whitening.** The series is whitened by the symmetric inverse square root $\hat{\Sigma}^{-1/2}$. Numerical verification shows the whitened sample covariance is at machine precision equal to I_p.
- **Autocovariances.** Estimated at lags 1 to 12 with $1/T$ normalization and no wrap-around. The antisymmetric part of the raw estimates is sampling noise and is removed by symmetrization. The symmetrized estimates agree with `acf()` to machine precision.
- **AMUSE.** One lag, base R `eigen()`, giving $\Lambda = \mathrm{diag}(U_\tau^\top \tilde{\Sigma}_\tau U_\tau)^2$, ordered decreasingly.
- **SOBI.** Lags 1 to 12, joint diagonalization via `JADE::rjd()`, giving $\Lambda = \sum_\tau \mathrm{diag}(U_\mathcal{T}^\top \tilde{\Sigma}_\tau U_\mathcal{T})^2$, ordered decreasingly. Orthogonality of V is at machine precision.
- **Verification.** An independent check with `JADE::SOBI()` confirms our SOBI solution agrees with the reference implementation up to sign and order.

The functions `center_series()`, `whiten_series()`, `autocov_raw()`, `symmetrize()`, `amuse_fit()`, and `sobi_fit()` defined in Section 1.3 will be reused in Part C, where each bootstrap sample must be re-centered, re-whitened, and re-diagonalized. Part B only requires the original data matrix `ts_data`.